

Render Engine

项目功能 · 架构思路 · 渲染原理 · C++ 工程实践

从 OpenGL 示例代码演进为中小型实时渲染引擎

C++11

OpenGL

PBR + IBL

Assimp

RenderGraph

项目阶段总结 / 2026.08



01 项目定位与能力边界

目标不是堆效果，而是建立可持续扩展的实时渲染框架

项目定位

学习型中小型实时渲染引擎

强调：资源复用、职责边界、可诊断性、渲染流程模块化

资产输入

OBJ / FBX / GLB / GLTF / DAE
外部贴图 + GLB Embedded Image
PBR 与旧材质兼容转换

画面能力

Metallic-Roughness PBR
IBL · HDR · Shadow · Point Light
Bloom 与多种屏幕后处理

工程能力

CMake + Visual Studio
Debug / Release 构建
运行时 DLL 自动部署

诊断能力

Shader 编译/链接日志
Framebuffer 完整性检查
OpenGL Debug Callback

工具能力

ImGui 参数面板
模型/贴图选择
后处理与曝光调节

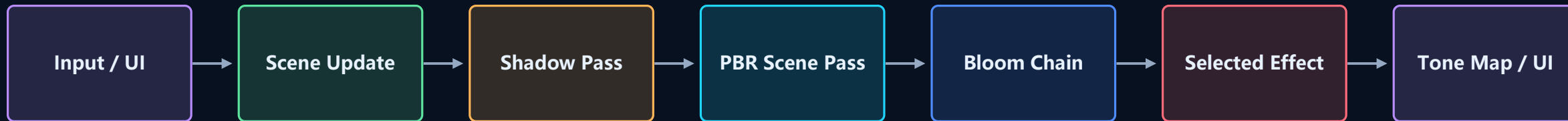
02 总体分层架构

核心原则：Scene 管世界，Renderer 管帧流程，AssetManager 管资源



03 一帧是怎样被渲染出来的

RenderGraph 负责组织 pass; RenderContext 负责提交底层命令



RenderGraph 记录 pass lambda → 保持顺序 → execute(RenderContext*)

CPU 组织

GPU 提交

当前是轻量顺序图; 未来升级为显式资源依赖 DAG

输出

HDR Color

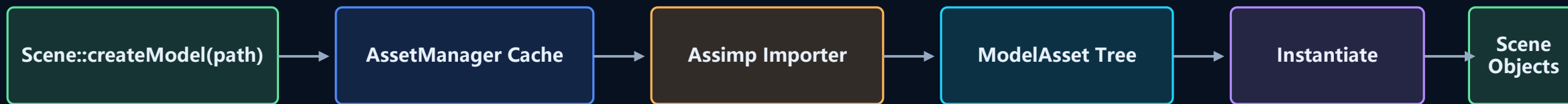
Depth / Shadow

Post Output

Backbuffer

04 模型与资源加载链路

导入数据、运行时实例和 GPU 资源逐步解耦



Asset (可缓存/共享)

ModelNodeAsset

- localTransform / children / mesh indices

MeshAsset

- CPU vertices / indices / flags

MaterialAsset

- ShaderHandle / factors / textures

Instance (场景状态)

GameObject / Transform

- parent-child hierarchy
- runtime TRS + source matrix

Mesh

- 当前实例的 VAO / VBO / IBO
- MaterialAsset 引用

下一步: MeshResource

同一个 ModelAsset 的多个实例目前仍重复上传 GPU Buffer。

下一阶段应缓存共享 VAO/VBO/IBO，让实例只保存资源句柄与 Transform。

Model Cache

Texture Cache

Embedded GLB

OBJ / FBX / GLTF / DAE

05 场景对象与 Transform

保留模型作者矩阵，同时允许运行时编辑实例

```
LocalMatrix = RuntimeTRS × SourceLocalMatrix  
WorldMatrix = ParentWorldMatrix × LocalMatrix
```

Source Matrix

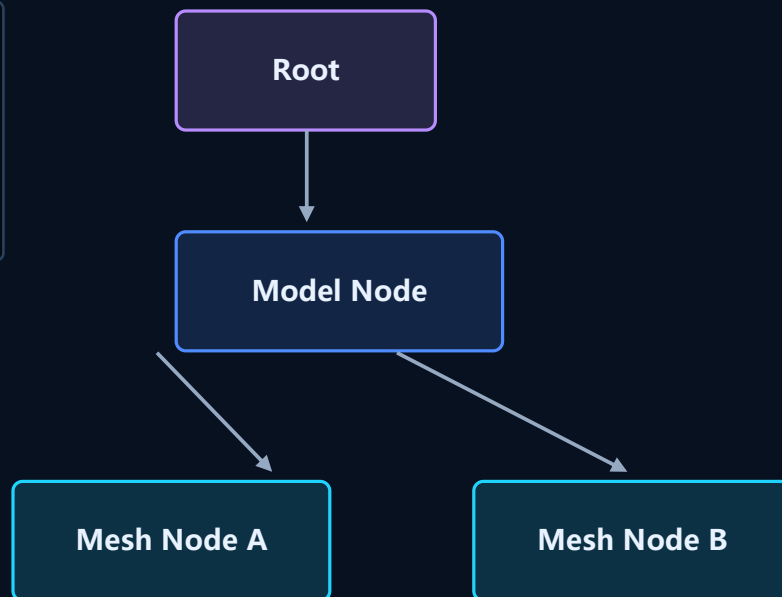
来自 FBX / glTF 节点层级

保存预旋转、缩放、坐标系转换
和 DCC authored transform

Runtime TRS

localPosition
localRotation
localScale

由游戏逻辑或编辑器修改



Scene 职责

- 拥有对象树
- 更新 world matrix
- 管理灯光
- 保存 Asset 引用
- 提供 renderable list

不负责:

Shader 选择、材质绑定、
pass 调度

骨骼动画接入后还需明确: $\text{Node Transform} \times \text{Animation Pose} \times \text{Inverse Bind Matrix}$

06 Material、Shader 与 Renderer 的职责边界

材质可以引用 Shader, 但不直接拥有 GL program 或执行 GL 调用



为什么这样拆分?

Shadow Pass 不需要绑定完整材质; Forward Pass 需要 PBR 数据。Renderer 根据 pass 决定流程, MaterialSystem 只负责数据映射。

类似 Unity 的地方

Unity Material 同样引用 Shader 和参数。区别在于引擎内部 Shader 资源由统一库管理, Material 保存的是句柄而不是裸 program。

可扩展方向

Technique / ShaderPass
Forward / Shadow / DepthOnly
Shader Variant / Keyword
Hot Reload

07 Metallic-Roughness PBR 原理

直接光使用微表面 BRDF, 间接光使用预计算 IBL

$$L_o = \int (kD \cdot \text{albedo} / \pi + D \cdot F \cdot G / 4(N \cdot V)(N \cdot L)) \cdot L_i \cdot (N \cdot L) d\omega$$

D • Normal Distribution

GGX / Trowbridge-Reitz
描述微表面法线分布, 高 roughness 产生更宽的高光。

G • Geometry

Smith + Schlick-GGX
描述微表面的遮蔽与阴影。

F • Fresnel

Schlick Approximation
控制视角相关反射; 金属 F0 来自 albedo。

材质输入

Base Color · Normal
Metallic · Roughness · AO
Emissive · Reflection Mask

glTF G = Roughness

glTF B = Metallic

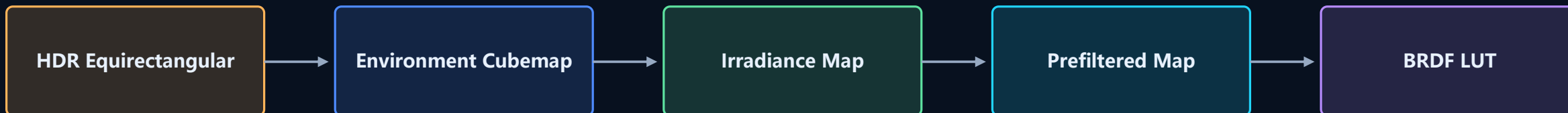
sRGB → Linear

HDR → Tone Map → Gamma

旧式材质: shininess → roughness 近似转换, 并继续读取 diffuse / normal / specular / AO / emissive。

08 IBL 预计算与运行时采样

把昂贵的环境积分前移到初始化阶段



Diffuse IBL

$N \rightarrow \text{Irradiance Map}$

低频环境辐照度 $\times$ albedo, 提供没有直接光时仍可信的环境漫反射。

Specular IBL

Reflection Vector + Roughness LOD

采样 Prefiltered Map, 不同 mip 对应不同模糊程度。

Split-Sum

Prefiltered Color $\times$ BRDF LUT

LUT 用 $N \cdot V$ 和 roughness 查找预积分 BRDF 系数。

初始化成本换取每帧性能：运行时只做少量纹理采样

09 灯光、阴影与后处理

光照解决“物理可信”，后处理解决“最终观感”

方向光 + Shadow Map

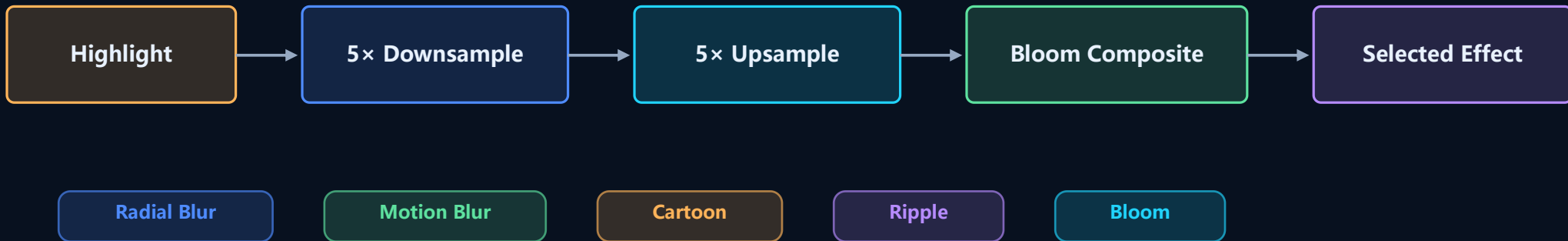
Depth-only Pass
Light Space Matrix
Polygon Offset + Bias
PCF Shadow Sampling
固定正交范围（待升级 CSM）

有限范围点光源

最多 4 个 Point Light
constant / linear / quadratic attenuation
range 平滑衰减，避免远距离染色
调试灯泡亮度与真实照明解耦

HDR 输出

RGBA32F Scene Target
曝光控制
Exponential Tone Mapping
sRGB Gamma Correction



10 本阶段升级: RenderTarget 资源化

将 FBO 描述、attachment 所有权和 resize 收敛到一个对象

BEFORE

Renderer 各自手工管理

FramebufferInfo
Texture2D* color / depth
Texture2D* bloom[5]
Texture2D* upsample[5]
resize() 手写列表
destructor 手写 delete
viewport 再从全局窗口计算

AFTER

RenderTarget 统一拥有

RenderTargetDesc
• format / sampler / clear action
• width / height / debug name

RenderTarget
• unique_ptr<Texture2D>
• FrameBufferInfo view
• resize / texture query
• non-copyable ownership

删除 10+ 裸纹理所有权字段

Bloom 使用实际 Mip 尺寸

显式注入 RenderContext&

为 RenderTargetPool 打基础

11 项目中的 C++ 核心知识点

语言特性不是孤立技巧，而是用来表达所有权、依赖和数据边界

RAII / unique_ptr

RenderTarget 独占 attachment
Renderer 独占子系统
析构自动释放 GPU wrapper
禁止复制表达唯一所有权

shared_ptr / Cache

ModelAsset、MaterialAsset、Texture2D 跨系统共享
AssetManager 保持缓存生命周期
实例只保存共享引用

引用与依赖注入

RenderContext& / AssetManager&
依赖必须存在但不归本对象所有
减少 singleton，提升可测试性

模板 + Lambda

RenderGraph::addPass<T, Fn>
保存不同参数和执行函数
统一通过虚函数 execute 调用

多态与抽象接口

RenderContext → OpenGLRenderContext
Light → Direction / Point
为替换后端和扩展类型保留边界

STL 与数据结构

vector: 对象/顶点/pass
unordered_map: 资源缓存
array: 固定 Bloom mip 链
move: 转移资源所有权

12 可诊断性与验证体系

目标：程序失败时直接看到原因，而不是只得到黑屏

日志系统

控制台 + logs/engine.log
INFO / WARN / ERROR
资源路径、缓存命中、模型统计
启动与关闭生命周期

GPU 错误定位

Shader 编译/链接 info log
源代码行号预览
Framebuffer completeness
OpenGL Debug Callback

构建部署

Debug / Release 双配置
Assimp DLL 自动复制
CMake 重新配置纳入新源码
运行时依赖检查

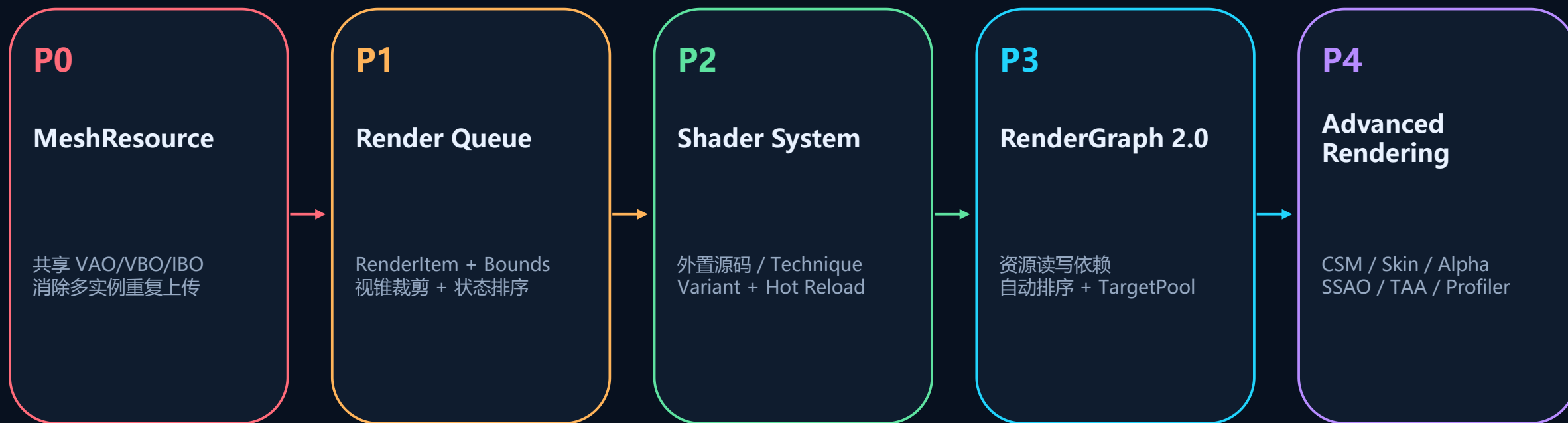
本轮验证

- Debug / Release 构建通过
- 默认场景与 Bloom 各运行 8 秒
- 14 个运行时 FBO 正常创建
- 1919×1009 → 800×600 连续 resize
- 无 Shader / FBO / 高优先级 GL 错误

已知非功能警告：驱动提示 Vertex Shader 会根据 GL State 重新编译；后续通过 Pipeline State 规范化继续优化。

13 当前问题与演进路线

下一阶段优先解决“共享 GPU 资源”和“渲染队列”，而不是继续增加孤立特效



架构债务

Scene/GameObject 裸指针 · RenderContext singleton · ShaderCode.cpp 硬编码 · CMake GLOB

画面能力缺口

CSM · glTF Alpha/Transmission/Skin · SSAO/TAA · GPU Profiler

Render Engine

从“能画出来”走向“能持续演进”

已经建立

资源缓存与 Asset 拆分
PBR + IBL + Shadow + HDR
Renderer / Scene / Material 边界
日志与运行时诊断

本轮推进

RenderTarget 统一所有权
后处理目标 RAI
Bloom 尺寸一致性
完整架构文档

下一目标

共享 GPU MeshResource
渲染队列与可见性
Shader Technique
RenderGraph 资源依赖

核心判断：架构的价值不是类更多，而是每个对象只负责一类变化。